

# Yousef Yousef

Event Booking · Weekly Mini Project 02 · Team 4 **Frontend**

**Responsibility:** Find / Manage Booking (Flow D), README and Demo Data

You own the screen where a person finds an existing booking and cancels it, plus the project README and the demo preparation. The Cancel button must follow what the backend says is allowed; the frontend never decides cancellation rules on its own.

## You provide

- The Find / Manage Booking screen
- The README and a ready demo script and data

## You depend on

- Allaith: project skeleton, routes, API client, types, shared components
- Adham: `POST /bookings/find` and `POST /bookings/:id/cancel` (cancel body is `{ phone }`)
- Majed: the seeded demo phone + code pairs

**Working rule:** all business rules are enforced in the backend. If something is unclear or the contract needs a change, tell the team leader (Allaith) so every file stays consistent.

## Your Flow, Phase by Phase

### Phase 1: Preparation

*Understand Flow D before building.*

- ☐ Read sections 6, 8 and 9 (Flow D) of the specification.
- ☐ Use Allaith's skeleton, API client, types and shared components.
- ☐ Use mock data matching the Booking type until Adham's endpoints are ready.

### Phase 2: Search Form

*Let the user look up a booking.*

- ☐ Route for Find / Manage Booking, reachable from the main navigation.
- ☐ Form with two required fields: phone number and booking code. Both must be filled before searching.
- ☐ Call `POST /bookings/find`. Disable the button while loading.
- ☐ Treat the code as case-insensitive in the UI (do not force the user to match capital letters).

### Phase 3: Booking Result

*Show what was found.*

- ☐ On success show: booking code, event title, date, time, location, customer name, phone, number of places, booking date and status badge (Active or Cancelled).
- ☐ On failure show the backend message clearly, such as "Booking not found. Check your phone number and booking code". Never show a blank page.
- ☐ Allow a new search without refreshing the page; clear the old result when a new search starts.

### Phase 4: Cancel Flow

*Cancel only when allowed and update immediately.*

- ☐ Show the Cancel button only when the booking's `canCancel` is true. Do not calculate this yourself.
- ☐ When `canCancel` is false, show a short reason instead: "This booking is already cancelled" or "This event has already started".
- ☐ Clicking Cancel opens a confirmation prompt ("Cancel this booking and return N places?").
- ☐ On confirm call `POST /bookings/:id/cancel` with `{ phone }`. Disable the button while sending.
- ☐ On success update the screen at once: status becomes Cancelled, the Cancel button disappears, and a message confirms the places were returned.
- ☐ If the backend rejects (already cancelled, event started), show its message exactly.

### Phase 5: Edge Cases and Testing

Verify the whole of Flow D with real data.

- ☐ Search with a correct pair: found. With a wrong phone, a wrong code, or both: not found.
- ☐ Cancel a booking, then check on the Events screen that the remaining places increased. Then book the same event again with the same phone and confirm it works.
- ☐ Cancel twice (second attempt must be impossible or rejected), and try a booking on the Past event (cancellation must not be offered).
- ☐ Search again for the cancelled booking: it must show Cancelled with no Cancel button.
- ☐ Coordinate with Allaith: optionally the Confirmation screen links here with the code already filled in.

### Phase 6: README and Demo Preparation

Make the project easy to run and the demo smooth.

- ☐ README sections: project description; requirements (Node version); how to install and start the backend; how to install and start the frontend; ports and the API URL setting; the fact that data is in memory and resets on backend restart.
- ☐ Document the seeded demo data: the Full, Almost full and Past events, and the demo phone + code pairs from Majed.
- ☐ Write a demo script, step by step: browse and filter, open an event, book, see the confirmation, see the updated remaining places, try an invalid booking, find the booking, cancel it, see places returned.
- ☐ Before every rehearsal, restart both servers and check the demo data is as expected.

### Phase 7: Presentation Preparation

Prepare your speaking part in English.

- ☐ Topic: the live demo of Find and Cancel (Flow D), including at least one invalid case rejected with a clear message.
- ☐ Rehearse the exact clicks and the exact phone + code you will type.

## Definition of Done

- ☐ Search needs both fields and shows clear "Booking not found" messages
- ☐ Result shows every required booking field and status
- ☐ Cancel button follows `canCancel` and updates the screen immediately
- ☐ Events screen shows places returned after cancelling
- ☐ README complete; demo script rehearsed with correct data

## Your Part in the Final English Presentation

Live demo of Find and Cancel (Flow D), including one invalid case.

## Appendix: API Contract Reference

| Route                  | Input                                                                    | Success        | Owner |
|------------------------|--------------------------------------------------------------------------|----------------|-------|
| GET /events            | query: category, date (YYYY-MM-DD), availableOnly=true, includePast=true | 200 + Event[]  | Majed |
| GET /events/categories | none                                                                     | 200 + string[] | Majed |
| GET /events/:id        | none                                                                     | 200 + Event    | Majed |
| POST /bookings         | body: eventId, customerName, customerPhone, places                       | 201 + Booking  | Adham |
| POST /bookings/find    | body: phone, code                                                        | 200 + Booking  | Adham |
|                        | body: phone                                                              |                | Adham |

|                               |  |                  |  |
|-------------------------------|--|------------------|--|
| POST /bookings/:id/<br>cancel |  | 200 +<br>Booking |  |
|-------------------------------|--|------------------|--|

**Event:** id, title, description, date (YYYY-MM-DD), time (HH:mm), location, category, capacity, booked, remaining, bookingPercentage, status

**status:** "Available" | "Almost full" | "Full" | "Past"

**Booking:** id, code (EV-XXXX), eventId, event (full Event, with updated availability), customerName, customerPhone, places, bookedAt, status ("Active" | "Cancelled"), canCancel (boolean)

**Error shape (every failure):** { statusCode, message }, where message is one readable sentence ready to show to the user. Codes: 400 invalid input, 404 not found, 409 rule conflict (full, not enough places, duplicate, past event, already cancelled, event started).

This contract is a proposal. The team leader confirms the final version; if a field name changes, the leader updates this reference.